

TESI DI LAUREA IN
INFORMATICA

Utilizzo di telecamere in ambienti industriali reali

CANDIDATO

Michele Ongaro

RELATORE

Prof. Agostino Dovier

TUTOR AZIENDALE

Ing. Umberto Piconi

CONTATTI DELL'ISTITUTO

Dipartimento di Scienze Matematiche, Informatiche e Fisiche

Università degli Studi di Udine

Via delle Scienze, 206

33100 Udine — Italia

+39 0432 558400

<https://www.dmif.uniud.it/>

Indice

1	Introduzione	1
1.1	Obiettivi	1
1.2	Presentazione scenario applicativo	1
1.3	Considerazioni implementative	2
2	Sviluppo delle metriche	3
2.1	Metriche basate sull'analisi delle frequenze	3
2.1.1	Fourier	3
2.1.2	Skewness	5
2.2	Metriche basate sull'edge detection	7
2.2.1	Laplacian	7
2.2.2	Canny	7
2.3	Altre metriche	8
3	Definizione dei threshold	11
4	Valutazione e confronto delle metriche	13
5	Conclusioni e considerazioni finali	15

1

Introduzione

1.1 Obiettivi

Si vuole realizzare un algoritmo che permetta di determinare se l'obiettivo di una telecamera sia sporco o meno tramite l'analisi delle immagini acquisite. In particolare, le immagini da considerare provengono da un flusso video ottenuto tramite delle telecamere IP posizionate in corrispondenza di una serie di targhe (*plate*) contenenti un codice realizzato tramite una codifica Hamming.

1.2 Presentazione scenario applicativo

L'infrastruttura aziendale presa in considerazione prevede una serie di telecamere IP, posizionate in punti strategici, raffiguranti delle cisterne contenenti materiale metallico fuso e con su appesa una targa (*plate*) contenente un certo codice. Tramite protocollo RTSP, le telecamere, forniscono un flusso di immagini ad una *pipeline* di riconoscimento di codici. Tali immagini vengono processate dalla *pipeline*, che attraverso una serie livelli di elaborazione riesce ad ottenere il valore numerico associato al codice del *plate*. Tuttavia, non sempre per la *pipeline* è possibile riconoscere tale valore, e questo è dovuto a due motivi principali. Il primo è dato dal fatto che il *plate* non è affatto presente nell'immagine o nell'area presa in considerazione dalla *pipeline*, infatti la procedura che porta al riconoscimento del valore numerico associato al *plate* può essere configurata in modo tale da considerare esclusivamente una sotto-area dell'immagine originale. Il secondo motivo per cui la *pipeline* non è capace di riconoscere il valore numerico è legato a fenomeni esterni come una scarsa illuminazione, una messa a fuoco dell'immagine precaria, la presenza parziale del *plate* o la stessa dimensione che il quest'ultimo assume nell'immagine complessiva. Un'altra casistica che porta la *pipeline* a fallire è la presenza di sporcizia sull'obiettivo, capita infatti che delle cisterne durante il loro movimento degli schizzi di metallo cadano sull'obiettivo della telecamera andandolo a sporcare. Il riconoscitore viene configurato con un certo limite di affidabilità e in questi casi succede che tale livello non può essere garantito dalla decodifica del codice e ciò comporta un esito negativo da parte dell'intera *pipeline*.

Per questo motivo è importante poter far affidamento ad un indice che sia indipendente dalla presenza del *plate* all'interno dell'immagine che permetta di valutare se l'obiettivo della telecamera sia in condizioni ottimali. In questo modo sarebbe possibile correggere la ripresa di una telecamera in antici-

po e prevenire lunghi periodi di tempo durante i quali la *pipeline* di riconoscimento non valuta alcun valore, ma non perché il codice è assente nell'immagine ma perché l'obiettivo della telecamera non ha la possibilità di fare delle riprese pulite.

1.3 Considerazioni implementative

Le telecamere sono configurate in modo tale da gestire la messa a fuoco in maniera automatica. Si è osservato che nei casi in cui l'obiettivo della telecamera viene sporcato da schizzi di metallo, la messa a fuoco cambia e prende come riferimento la macchia presente sull'obiettivo, questo ha come effetto che la restante porzione dell'immagine diventi completamente sfocata e la *pipeline* di riconoscimento non garantisce più un adeguato livello di affidabilità. Per questo motivo lo studio che verrà illustrato di seguito si basa esclusivamente sulla considerazione di immagini sfocate / nitide. Se quindi l'algoritmo riuscirà a trovare che una immagine è sfocata allora questo rappresenterà un indice chiaro del fatto che la telecamera è fuori fuoco e la causa di ciò sarà con buona probabilità semplicemente dovuto al fatto che l'obiettivo è stato sporcato.

2

Sviluppo delle metriche

Per ottenere un indice che permetta di determinare la nitidezza di una immagine si può procedere facendo uso di una serie di approcci diversi. Si può fare distinzione tra le **metriche basate sull'analisi delle frequenze** e le **metriche basate sull'edge detection**. Queste sono le due tipologie principali utilizzate nell'ambito del *image processing* ma non sono le sole, infatti si può andare ad osservare ulteriori caratteristiche della natura dell'immagine che non sono, almeno direttamente, associabili ad una di queste due macro tipologie. Prima di eseguire uno qualsiasi di questi algoritmi l'immagine subirà un *preprocessing* che andrà a normalizzare alcune caratteristiche di essa, in particolare lo studio è che stato effettuato ha considerato una normalizzazione della luminosità e una conversione dei colori in scala di grigio.

2.1 Metriche basate sull'analisi delle frequenze

L'idea che sta alla base dell'utilizzo di una metrica basata sull'analisi delle frequenze, sta nel considerare una immagine come un segnale bidimensionale. Le immagini infatti sono dei segnali discreti a due dimensioni. Il valore assunto da ciascun *pixel* è paragonabile al valore assunto dall'ordinata in un segnale monodimensionale. Se quindi, utilizzando strumenti come la trasformata di Fourier, è possibile estrarre lo spettro delle frequenze, allora si può constatare che una immagine nitida, rispetto ad una sfocata, sarà composta da frequenze più elevate. Tali frequenze nell'immagine vengono rappresentate dai piccoli dettagli o dalle rapide variazioni di colore, caratteristiche invece non presenti in una immagine sfocata.

Entrambe le metriche che seguono si basano sulla trasformata discreta di Fourier (DTF) e differiscono nel modo in cui tale risultato viene valutato.

2.1.1 Fourier

Questa metrica sfrutta il posizionamento dei valori della matrice di Fourier [6] per determinare le frequenze basse e alte presenti nell'immagine. Dopo aver computato la matrice di Fourier, avremo a disposizione dei numeri complessi che rappresentano lo spettro dell'immagine considerata. In posizione centrale a questa matrice troveremo la frequenza 0, e mano a mano che ci spostiamo sulla periferia avremo a disposizione le frequenze più basse. Per ognuno di questi numeri complessi andiamo a calcolare il

modulo e lo andiamo a salvare in una matrice di dimensioni analoghe. Consideriamo poi la sommatoria delle frequenze basse e alte considerando "basse" le frequenze che si trovano all'interno di un cerchio avente raggio pari al 10% del valore minore tra altezza e larghezza della matrice di Fourier, le frequenze "alte" saranno tutte le altre. L'indice α poi considererà il rapporto che c'è tra queste due sommatorie:

$$\alpha = \frac{\sum_{f \in \Omega_{low}} ||f||}{\sum_{f \in \Omega} ||f||} \quad (2.1)$$

Dove α è il *blur value*, Ω è la trasformata di fourier e Ω_{low} sono i valori della trasformata di fourier per le frequenze più basse.

α è un valore che sta tra 0 e 1, se il denominatore è più grande rispetto al numeratore significa che saranno presenti contributi significativi dalle frequenze più alte e quindi l'immagine sarà più nitida e il *blur value* si muoverà verso lo 0, via via che l'immagine diventa più sfocata i contributi delle frequenze più alte diminuiranno e il rapporto si avvicinerà a 1.

Lo pseudo codice che implementa questa metrica è il seguente:

```
1 def getBlurValue(image):
2     omega = fft(image)           # trasformata di fourier
3     magnitude = abs(omega)       # calcolo moduli
4     h, w = shape(magnitude)
5     cutoff = min(h, w) * 0.1    # limite per le basse frequenze
6
7     low = magnitude[where r < cutoff].sum()
8     high = magnitude[where r >= cutoff].sum()
9
10    return low / (high + low)
```

Le immagini che seguono sono una rappresentazione della trasformata di Fourier di due immagini che differiscono nella loro nitidezza. Si può notare che nella prima il maggior valore dei moduli si trova nel centro, dove vengono posizionate le frequenze più basse, al contrario nella seconda osserviamo moduli più alti in corrispondenza delle frequenze più alte.

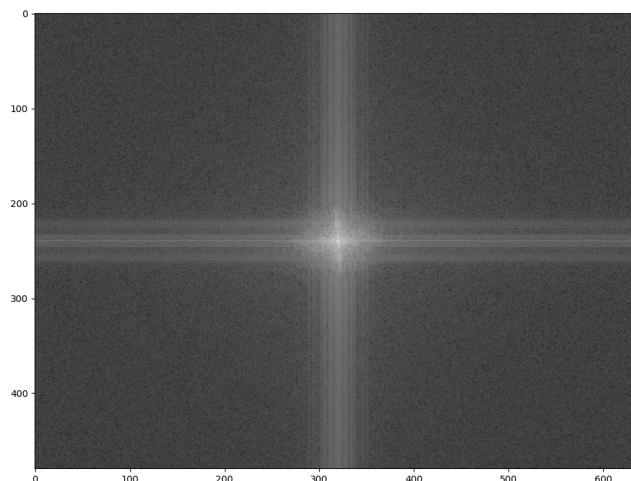


Figura 2.1: Magnitude della FFT per una immagine sfocata

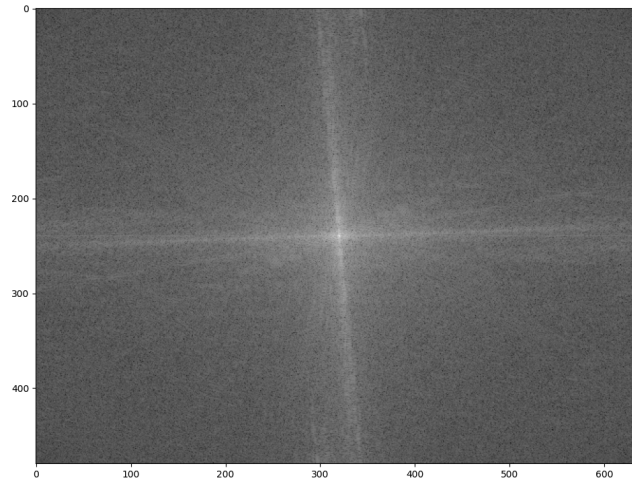


Figura 2.2: Magnitude della FFT per una immagine nitida

2.1.2 Skewness

Questa metrica fa uso dell'indice di *skewness* [2], ossia un valore che permette di misurare la forma della funzione di distribuzione delle frequenze. Per costruire la curva di distribuzione vanno considerati i valori assunti dai moduli degli elementi della trasformata di Fourier andando a campionare i valori in cerchi concentrici centrati nella frequenza 0. L'indice di *skewness* ci dirà poi se la distribuzione è *right-skewed* con un numero positivo o *left-skewed* con un numero negativo. In particolare ci aspettiamo una distribuzione *right-skewed* siccome i contributi delle basse frequenze contribuiscono in maniera più significativa, specialmente quando l'immagine diventa più sfocata. L'indice di *skewness* sarà poi ottenuto tramite una formula, come ad esempio quella di *Pearson*:

$$Pearson's\ median\ skewness = 3 \cdot \frac{Mean - Median}{Standard\ deviation} \quad (2.2)$$

Come si può constatare dalle immagini che seguono, l'idea è quella che un'immagine sfocata ha una curva *right-skewed* più accentuata rispetto a quella di una immagine più nitida, quindi questa metrica darà un valore più alto che sta a significare che la curva si sta muovendo sempre di più da una distribuzione normale.

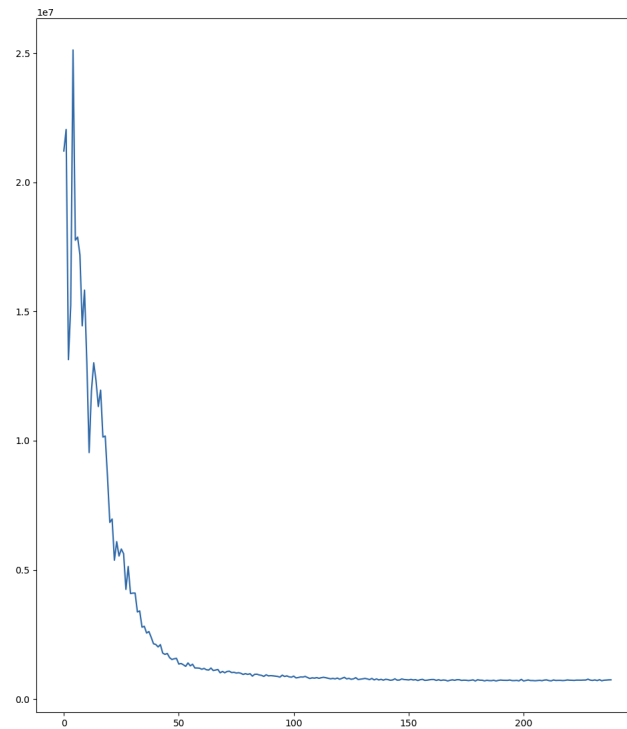


Figura 2.3: Somma magnitudini per frequenza di una immagine sfocata

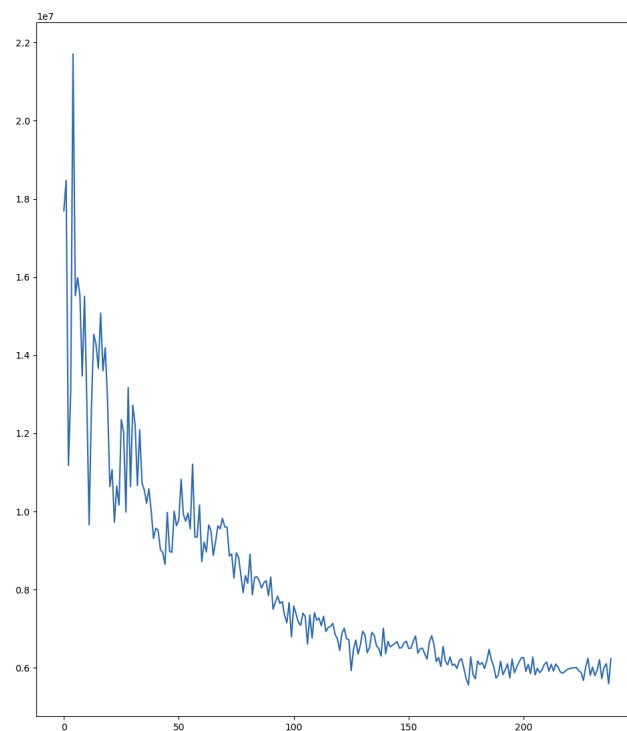


Figura 2.4: Somma magnitudini per frequenza di una immagine nitida

Dopo aver calcolato l'indice di *skewness* per normalizzare il risultato e mantenerlo all'interno dell'intervallo $[0, 1]$ è possibile applicare la funzione sigmoide. Lo pseudocodice che descrive il calcolo per questa metrica è quindi il seguente:

```

1 def getBlurValue(image):
2     omega = fft(image)
3     magnitude = abs(omega)
4     h, w = shape(fft)
5
6     radiusValues = []
7     for (radius = 1, radius < min(w,h), radius++):
8         radiusValues.push(magnitude[r == radius].sum())
9
10    mean = mean(radiusValues)
11    median = median(radiusValues)
12    std = std(radiusValues)
13    if std == 0:
14        return 1
15    skew = 3 * (mean - median) / std
16    return sigmoid(skew)

```

2.2 Metriche basate sull'edge detection

Per quanto riguarda le metriche basate sull'*edge detection* il concetto alla base delle diverse interpretazioni è che un'immagine nitida ha dei bordi ben definiti e in quantità maggiore rispetto ad un'immagine sfocata.

2.2.1 Laplacian

Questa metrica prevede l'utilizzo del filtro Laplacian [3]. Questo filtro permette di calcolare la derivata spaziale di una immagine e lo fa rilevando le regioni dell'immagine che subiscono dei cambiamenti di intensità immediati. Una volta ottenuta la derivata, valutandone la sua varianza sarà possibile creare un indice inversamente proporzionale al livello di sfocatura: se la varianza è elevata allora l'immagine sarà nitida e questo deriva dal fatto tale tipo di immagini possiede solitamente dei cambiamenti piuttosto rapidi dei valori assunti dai vari pixel.

```

1 def getBlurValue(image):
2     edges = laplacian(image)
3     if edges.var() == 0:
4         return 1
5     return 1 / edges.var()

```

2.2.2 Canny

In maniera completamente analoga alla metrica Laplacian appena vista, con l'operatore Canny [4] è possibile ricavare una particolare immagine di output che mostra le discontinuità di intensità rilevate. Canny si suddivide in più fasi. Inizialmente all'immagine di partenza viene applicata una convoluzione Gaussiana per rimuovere il rumore, ossia le frequenze più alte, poi tramite un filtro di derivata spaziale come Sobel [5] o analoghi esegue il calcolo della derivata prima. Gli spigoli quindi danno origine alle creste presenti nell'immagine gradiente, queste creste vengono prese in considerazione dall'algoritmo che imposta a 0 i pixel che non hanno come valore di cresta massimale. La metrica che poi va a rilevare

la sfocatura dell'immagine considera la varianza dell'immagine risultante, questo valore è direttamente proporzionale alla nitidezza ma inversamente proporzionale alla sfocatura perciò se la varianza è nulla il valore di sfocatura (*blurValue*) sarà 0, il che vuol dire che l'immagine è completamente sfocata, se però la varianza è strettamente maggiore allora la metrica ne calcolerà il reciproco in modo che la metrica possa crescere insieme al livello di sfocatura dell'immagine

```
1 def getBlurValue(image):
2     edges = canny(image)
3     if edges.var() == 0:
4         return 1
5     return 1 / edges.var()
```

2.3 Altre metriche

Le metriche basate sull'analisi delle frequenze e le metriche basate sull'edge detection non sono le sole, infatti è possibile creare un indice per stabilire se l'immagine è sfocata o meno anche osservando la dimensione in byte dell'immagine JPEG [1]. Le immagini JPEG sfocate infatti occupano poco spazio perché la compressione JPEG è basata sulla perdita di dettaglio ad alta frequenza, che sono esattamente quelli che definiscono la nitidezza e i bordi definiti. Per una immagine sfocata il processo di compressione ha meno informazioni complesse da eliminare o ridurre, risultando in file più piccoli rispetto ad una immagine completamente nitida. Questa soluzione ha il vantaggio di essere molto semplice ma bisogna tenere a mente che è una metrica molto delicata siccome due immagini nitide con stessa risoluzione potrebbero avere delle dimensioni completamente diverse, questo perché la dimensione dipende da altri fattori come ad esempio il range di colori usato nell'immagine, o la dimensione che i dettagli assumono all'interno dell'immagine. Nel grafico seguente viene riportato l'andamento della dimensione di una immagine JPEG 640x480 che viene via via sfocata in maniera sempre più accentuata attraverso un kernel di dimensione sempre maggiore.

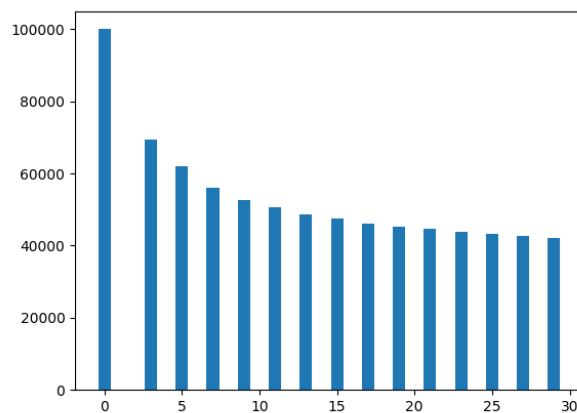


Figura 2.5: Dimensione di una immagine JPEG che viene via via sfocata

Un'altro sistema che si può utilizzare per costruire una metrica che permetta di capire se una immagine è sfocata o no consiste nell'analisi della varianza dell'immagine stessa. L'immagine infatti non è

altro che un vettore bidimensionale contenente dei valori numerici che vanno a definire il colore di ogni pixel e la varianza è direttamente proporzionale alla nitidezza dell'immagine. Un'immagine monocromatica, ossia l'immagine sfocata per antonomasia, avrà una varianza nulla e via via che aggiungiamo dettagli all'immagine otterremo una varianza sempre maggiore.

3

Definizione dei threshold

Le metriche che sono state definite offrono un indice numerico che può venire usato per determinare il livello di *blur* di una immagine, ma per stabilire se una immagine sia sfocata o meno bisogna stabilire un limite (*threshold*) al di sopra del quale si può dire con buona approssimazione che l'immagine sia sfocata, e idealmente al di sotto del quale l'immagine è ancora nitida.

Ogni metrica avrà una soglia diversa siccome si basano su approcci, come è stato mostrato, diversi ma sicuramente sarà un valore t compreso nell'intervallo $[0, 1]$. Avremo quindi che $t_{fourier}$ sarà diverso da t_{skew} che a sua volta sarà diverso da $t_{laplacian}$ e così via. In pratica ogni *threshold* t dovrà venire considerato in relazione ad una certa metrica. Supponendo di aver scelto una certa metrica il valore t potrà essere determinato tramite i valori $t_1, t_2, \dots, t_N$ dove N è il numero di telecamere presenti nell'infrastruttura aziendale. Ogni telecamera quindi contribuirà al risultato finale offrendo un *threshold* locale t_i che a sua volta sarà determinato dallo studio delle immagini acquisite da tale telecamera. Supponiamo che l'infrastruttura aziendale offra un numero pari a 3 telecamere. Avremo che $N = 3$ e quindi avremo a disposizione i valori t_1, t_2, t_3 che sono rispettivamente i *threshold* da utilizzare per capire se una immagine è sfocata rispettivamente per la telecamera 1, 2 e 3. Il risultato generale t potrà poi essere ottenuto dalla media:

$$t = \frac{t_1 + t_2 + t_3}{3} \quad (3.1)$$

Più in generale per un numero N di telecamere avremo che:

$$t = \frac{1}{N} \sum_{i=1}^N t_i \quad (3.2)$$

Supponiamo che ogni telecamera i metta a disposizione un certo numero di immagini n_i . Consideriamo la telecamera 10 per esempio, il numero di immagini messe a disposizione da tale ripresa sarà n_{10} . Ogni telecamera metterà quindi a disposizione un certo numero di n_i di *funzioni immagine*. Una *funzione immagine* accetta come parametro la dimensione del kernel da usare per sfocare l'immagine e restituisce l'immagine sfocata con il kernel scelto. Quindi $I_i^j(1)$ rappresenta la j -esima immagine (con $1 \leq j \leq n_i$) della telecamera i (con $1 \leq i \leq N$) sfocata con un kernel di dimensione 1, ossia la j -esima immagine ripresa dalla telecamera i nella sua condizione originale. Se volessimo ottenere la 3^a immagine della telecamere 10 sfocata con un kernel di dimensione 29 calcoleremo: $I_{10}^3(29)$. Si ricorda che la

dimensione del kernel va fornita tramite un numero dispari siccome la matrice kernel che applicherà la sfocatura tramite un filtro gaussiano, viene applicata tramite una operazione di convoluzione, in altre parole è necessario avere un valore centrale della matrice siccome questo verrà centrato sul pixel su cui verrà applicata l'operazione. La definizione di questa particolare funzione è importante siccome servirà a formalizzare un modo per ottenere i diversi t_i .

A questo punto per ogni telecamera bisognerà considerare le diverse immagini, ogni immagine infatti fornirà un *threshold* indicizzato da t_i^j dove i è il numero della telecamera $1 \leq i \leq N$ e j è il numero dell'immagine $1 \leq j \leq n_i$. Per ottenere t_i^j bisogna calcolare il risultato di $I_i^j(1), I_i^j(3), I_i^j(5) \dots$ e per ognuna di queste immagini via via sempre più sfocate verificare se il riconoscitore riesce o meno a identificare il numero codificato dal *plate*. A partire da una certa dimensione del kernel si osserverà che il riconoscitore fallirà in continuazione, sia k_i^j la dimensione del kernel per cui da $k_i^j + 1$ in poi il riconoscitore darà sempre esito negativo i.e. le immagini $I_i^j(k_i^j + 1), I_i^j(k_i^j + 2), \dots$ non saranno nitide a sufficienza per poter riconoscere il codice del *plate*. Per ottenere t_i^j sarà sufficiente calcolare il *blur value* dell'immagine $I_i^j(k_i^j + 1)$:

$$t_i^j = \alpha_{I_i^j(k_i^j+1)} \quad (3.3)$$

dove α_ξ è il *blur value* calcolato con la metrica scelta per l'immagine ξ .

Successivamente per ottenere il valore di t_i bisognerà combinare i vari t_i^j , una media può andare bene ma altre funzioni come ad esempio la mediana si possono usare se si reputa che nel *dataset* ci siano dei casi particolari che altererebbero in maniera significativa il risultato.

$$t_i = \frac{1}{n_i} \sum_{j=1}^{n_i} t_i^j \quad (3.4)$$

In questo modo possiamo andare a definire dei t diversi per ogni metrica, che possiamo chiamare $t_{fourier}$, t_{skew} , $t_{laplacian}$ e t_{canny} e saranno definiti complessivamente dalla formula generica:

$$t_m = \frac{1}{N} \sum_{i=1}^N \left(\frac{1}{n_i} \sum_{j=1}^{n_i} \alpha_{I_i^j(k_i^j+1)}^m \right) \quad (3.5)$$

dove m è la metrica di scelta e α_ξ^m rappresenta il *blur value* dell'immagine ξ calcolata tramite la metrica m .

4

Valutazione e confronto delle metriche

Ogni metrica porterà alla definizione di un certo t , tale risultato deve essere ottenuto

5

Conclusioni e considerazioni finali

Bibliografia

- [1] CCITT. *T.81 – DIGITAL COMPRESSION AND CODING OF CONTINUOUS-TONE STILL IMAGES – REQUIREMENTS AND GUIDELINES*. International Telecommunication Union, 1992.
- [2] N. Balakrishnan NL. Johnson, S. Kotz. *Continuous Univariate Distributions. Vol. 1 (2 ed.)*. Wiley, 1994.
- [3] L. Shapiro R. Haralick. *Computer and Robot Vision pp 346-351*. Addison-Wesley, 1992.
- [4] Zhang Wei Sun Lining Rong Weibin, Li Zhanjing. *An improved Canny edge detection algorithm, pp 577-582*. IEEE, 2014.
- [5] Irwin Sobel. *History and Definition of the Sobel Operator*. 2014.
- [6] Gilbert Strang. *Wavelets*. American Scientist, Volume 82, Issue 3, 1994.